



CSE 203: Data Structure

Data
Structure

Character:

=> Non numerical data.

=> Are represented in memory by 8 bits

Alphabet:	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Digits:	0 1 2 3 4 5 6 7 8 9
Special characters:	+ - / * () , . \$ = ' □



Character:

- => Non numerical data.
- => Are represented by 8 bits

String:

- => Sequence of characters.
- => A finite sequence of zero or more characters
- => Working on string is referred by string processing/string manipulation/ text editing.

Example of string:

Let four string variables S1, S2, S3, S4 and S5 where:

S1= 'CSE'

S2='JKKNIU'

S3=' MYMENSINGH'

S4='□'

S5=""



Character:

=> Non numerical data.

=> Are represented by 8 bits

Example of string:

Let five string variables S1, S2, S3, S4 and S5 where:

S1= 'CSE'

S2='JKKNIU'

S3=' MYMENSINGH'

S4='□'

S5=""

Length of String:

length of S1 is 3

length of S2 is 6

length of S3 is 10

length of S4 is 1

length of S5 is 0

Length:

- Number of characters in a string is its length.
- String with zero characters is called empty/null string, ex. S5.



A string Y is called a *substring* of a string S if there exist strings X and Z such that

$$S = X//Y//Z$$

If X is an empty string, then Y is called an *initial substring* of S , and if Z is an empty string then Y is called a *terminal substring* of S . For example,

'BE OR NOT' is a substring of 'TO BE OR NOT TO BE'
'THE' is an initial substring of 'THE END'

Clearly, if Y is a substring of S , then the length of Y cannot exceed the length of S .



Storing String:

Three Structural Methods:

1. Fixed length structure
2. Variable length structure
3. Linked structure

1. Fixed length structure

Each line of print is viewed as a record.

All records have the same length.



CSE 203: Data Structure

Data
Structure

Storing String:

1. Fixed length structure

Each line of print is viewed as a record.

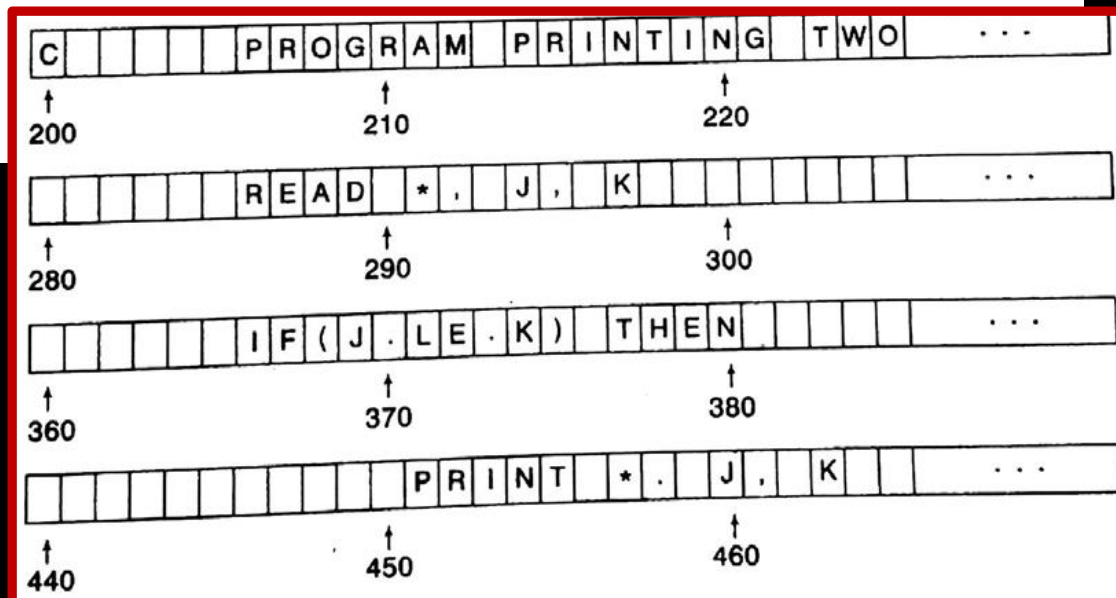
All records have the same length.

Consider the following program

```
Rec 1: C PROGRAM PRINTING TWO INTEGERS IN INCREASING ORDER
Rec 2: READ *, J, K
.... IF(J.LE.K) THEN
.... PRINT *, J, K
ELSE
PRINT *, K, J
ENDIF
STOP
Rec 9: END
```

To store the program

When the length of record is 80

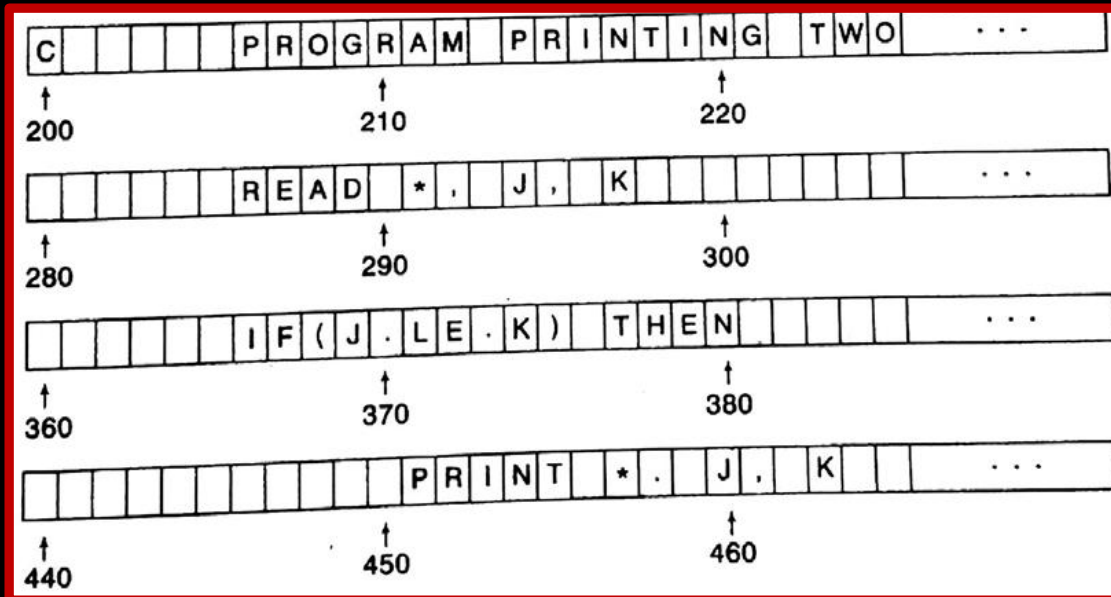




CSE 203: Data Structure

Data
Structure

Storing String: 1. Fixed length structure



The main disadvantages are:

- (1) Time is wasted reading an entire record if most of the storage consists of inessential blank spaces.
- (2) Certain records may require more space than available.
- (3) When the correction consists of more or fewer characters than the original text, changing a misspelled word requires the entire record to be changed.

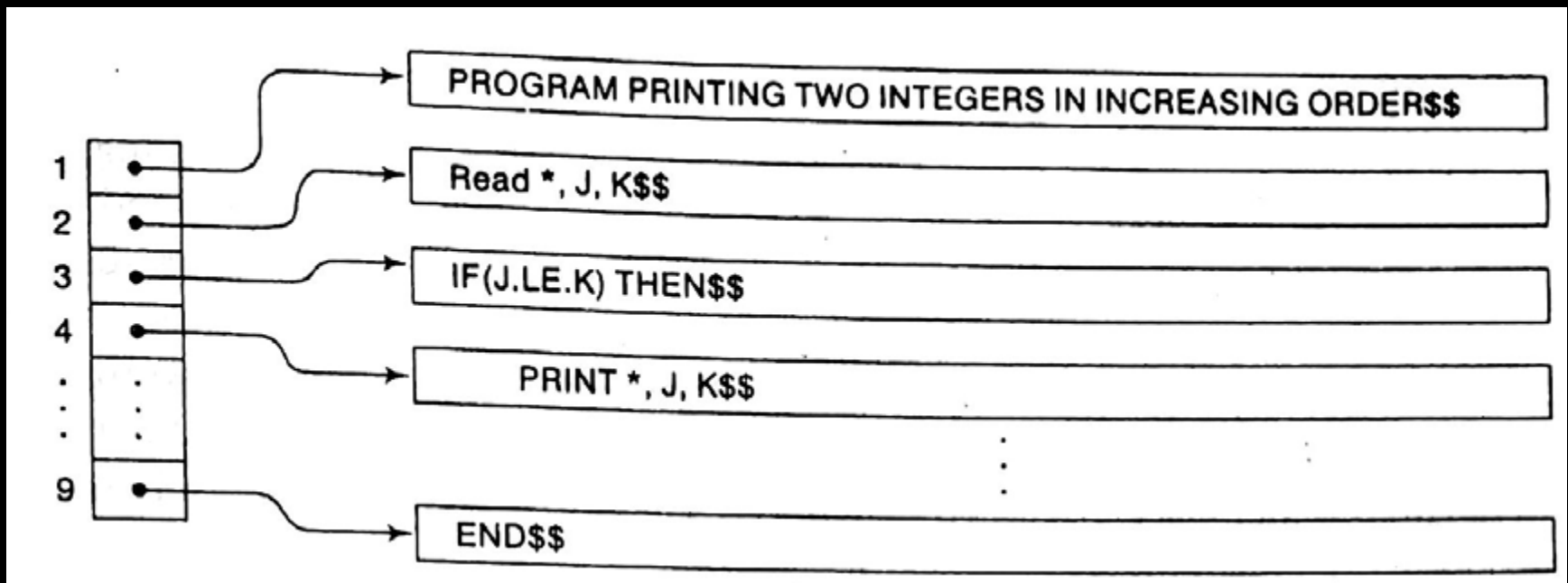


CSE 203: Data Structure

Data
Structure

Storing String: 2. Variable length structure

One can use a marker at the end, For example \$\$



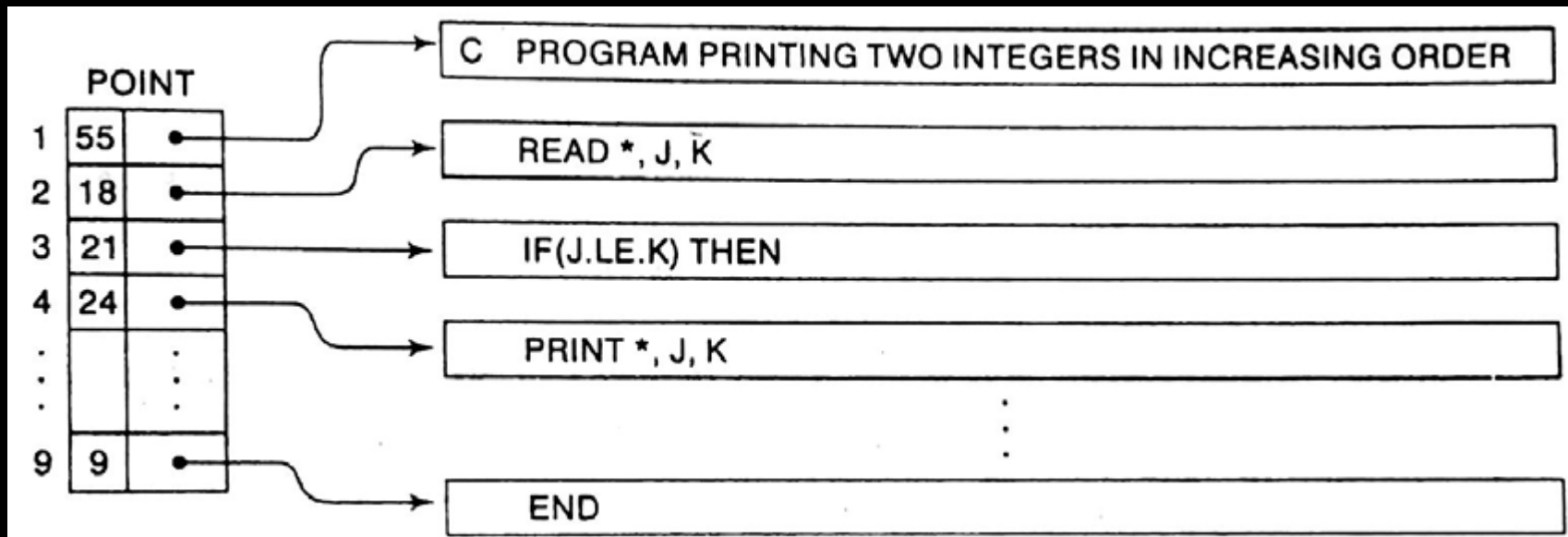


CSE 203: Data Structure

Data
Structure

Storing String: 2. Variable length structure

One can use the length of the string

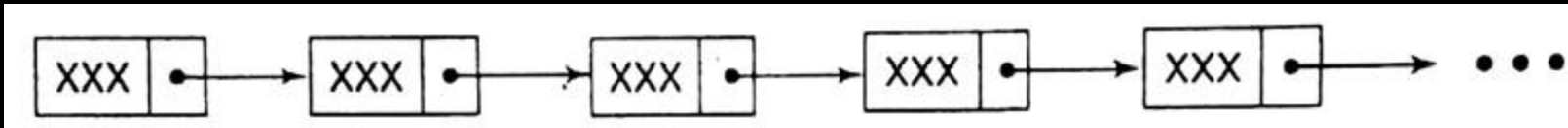




CSE 203: Data Structure

Data
Structure

Storing String: 3. Linked structure





String Operations:

1. Substring: Group of consecutive elements in a string

Substring (string, initial, length)

Substring ('CSE, JKKNUIU', 1, 3) =>

CSE

Substring ('CSE, JKKNUIU', 3, 5) =>

E, JK

String: 'CSE, JKKNUIU'

Substring: 'CSE', 'JKKNUIU'

```
int main()
{ char string[1000] ={"CSE, JKKNUIU"}, sub[1000];
  int position=3, length=5, c = 0;
  while (c < length)
  { sub[c] = string[position+c-1];
    c++;
  }
  sub[c] = '\0';

  printf("Required substring is \"%s\"", sub);
  return 0;
}
```



String Operations:

2. Indexing/ Pattern matching:

Where a pattern P is found in a text T

INDEX (Text, Pattern)

T='HIS FATHER IS THE PROFESSOR' P='THE'

INDEX (T, P)

At index 7



String Operations:

2. Indexing/ Pattern matching:

Where a pattern P

INDEX (Text, Pattern)

T='HIS FATHER IS THE

INDEX (T, P)

```
void Index(char* txt, char* pat)
{
    int M = strlen(pat);
    int N = strlen(txt);
    for (int i = 0; i <= N - M; i++) {
        for (int j = 0; j < M; j++)
            if (txt[i + j] != pat[j])        break;

        if (j == M) // if pat[0...M-1] = txt[i, i+1, ...i+M-1]
            printf("Pattern found at index %d ", i);
    }
}
```

```
int main()
{
    char txt[] = "AABAACAADAABAAABAA";
    char pat[] = "AABA";
    Index(txt, pat);
    return 0;
}
```



String Operations:

2. Indexing/ Pattern matching:

Where a pattern P

INDEX (Text, Pattern)

T='HIS FATHER IS TH

INDEX (T, P)

M=4

N=10

N-M=6

When $i=0, j=0$
 $\text{text}[0] == \text{pat}[0]$

When $i=0, j=1$
 $\text{text}[1] != \text{pat}[1]$
Break j loop

```
void search(char* pat, char* txt)
{
    int M = strlen(pat);
    int N = strlen(txt);
    for (int i = 0; i <= N - M; i++) {
        for (int j = 0; j < M; j++)
            if (txt[i + j] != pat[j]) break;

        if (j == M) // if pat[0...M-1] = txt[i, i+1, ...i+M-1]
            printf("Pattern found at index %d ", i);
    }
}

int main()
{
    char txt[] = "ABAAABABAA";
    char pat[] = "AABA";
    search(pat, txt);
    return 0;
}
```



CSE 203: Data Structure

Data
Structure

String Operations:

2. Indexing/ Pattern matching:

Where a pattern P

INDEX (Text, Pattern)

T='HIS FATHER IS TH

INDEX (T, P)

M=4

N=10

N-M=6

When i=1

For j=0, text[1]!=pat[0]

Break j loop

```
void search(char* pat, char* txt)
```

```
{
```

```
    int M = strlen(pat);
```

```
    int N = strlen(txt);
```

```
    for (int i = 0; i <= N - M; i++) {
```

```
        for (int j = 0; j < M; j++)
```

```
            if (txt[i + j] != pat[j])        break;
```

```
        if (j == M) // if pat[0...M-1] = txt[i, i+1, ...i+M-1]
```

```
            printf("Pattern found at index %d ", i);
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    char txt[] = "ABAAABABAA";
```

```
    char pat[] = "AABA";
```

```
    search(pat, txt);
```

```
    return 0;
```

```
}
```



String Operations:

2. Indexing/ Pattern matching:

Where a pattern P

INDEX (Text, Pattern)

T='HIS FATHER IS THE

INDEX (T, P)

M=4

N=10

N-M=6

When i=2,

For j=0, text[2]==pat[0]

For j=1, text[3]==pat[1]

For j=2, text[4]!=pat[2]

Break j loop

```
void search(char* pat, char* txt)
{
    int M = strlen(pat);
    int N = strlen(txt);
    for (int i = 0; i <= N - M; i++) {
        for (int j = 0; j < M; j++)
            if (txt[i + j] != pat[j]) break;

        if (j == M) // if pat[0...M-1] = txt[i, i+1, ...i+M-1]
            printf("Pattern found at index %d ", i);
    }
}

int main()
{
    char txt[] = "ABAAABABAA";
    char pat[] = "AABA";
    search(pat, txt);
    return 0;
}
```



String Operations:

2. Indexing/ Pattern matching:

Where a pattern P

INDEX (Text, Pattern)

T='HIS FATHER IS TH

INDEX (T, P)

M=4

N=10

N-M=6

When i=3,

For j=0, text[3]==pat[0]

For j=1, text[4]==pat[1]

For j=2, text[5]==pat[2]

For j=3, text[6]==pat[3]

Then J=4

```
void search(char* pat, char* txt)
{
    int M = strlen(pat);
    int N = strlen(txt);
    for (int i = 0; i <= N - M; i++) {
        for (int j = 0; j < M; j++)
            if (txt[i + j] != pat[j]) break;

        if (j == M) // if pat[0...M-1] = txt[i, i+1, ...i+M-1]
            printf("Pattern found at index %d ", i);
    }
}

int main()
{
    char txt[] = "ABAAABABAA";
    char pat[] = "AABA";
    search(pat, txt);
    return 0;
}
```



String Operations:

2. Indexing/ Pattern matching:

Where a pattern P

INDEX (Text, Pattern)

T='HIS FATHER IS THE

INDEX (T, P)

M=4

N=10

N-M=6

When i=3,

For j=0, text[3]==pat[0]

For j=1, text[4]==pat[1]

For j=2, text[5]==pat[2]

For j=3, text[6]==pat[3]

Then J=4

```
void search(char* pat, char* txt)
{
    int M = strlen(pat);
    int N = strlen(txt);
    for (int i = 0; i <= N - M; i++) {
        for (int j = 0; j < M; j++)
            if (txt[i + j] != pat[j])        break;

        if (j == M) // if pat[0...M-1] = txt[i, i+1, ...i+M-1]
            printf("Pattern found at index %d ", i);
    }
}

int main()
{
    char txt[] = "ABAAABABAA";
    char pat[] = "AABA";
    search(pat, txt);
    return 0;
}
```



CSE 203: Data Structure

Data
Structure

String Operations:

2. Indexing/ Pattern matching:

Where a pattern P

INDEX (Text, Pattern)

T='HIS FATHER IS THE

INDEX (T, P)

Another method

```
main()
{
    char A[]="ABABAABABC", B[]="AABA", C[10];
    int i, j, k, L1,L2;
    L1=strlen(A);
    L2=strlen(B);
    for(i=0;i<(L1-L2);i++)
    {
        for(j=0;j<L2;j++)
            C[j]=A[i+j];
        C[j]='\0';
        k=strcmp(B,C);
        if(k==0) break;
    }
    if(k==0) printf("Pattern is found at %d",i);
    else printf("Not matching");
    getch();
}
```



String Operations:

3. Concatenation of String: Let S1 and S2 be two individual string. S1='CSE' S2='JKKNIU'
- Consisting of S1 and S2 is the string consisting of the character of S1 followed by the characters of S2. Denoted by S1//S2.

```
main()
{ char A[]="CSE, ", B[]="JKKNIU", C[20], D[20];
//Method 1
C=strcat(A,B);
printf("%s",C);

//Method 2
for(i=0;i<strlen(A);i++)
    D[i]=A[i];

For(j=i; j<(i+strlen(B)); j++)
    D[j]=B[j-i];
D[j]='\0';
Printf("%s", D)
}
```



4. Length:

Length

The number of characters in a string is called its length. We will write
`LENGTH(string)`

C command:

First include string header file using

```
#include<string.h>
```

```
Char S[]="JKKNIU";
```

Method 1:

```
Int L=strlen(S);
```

```
printf("%d", L)
```

Method 2:

```
int L=0;
```

```
while(S[L]!='\0') L++;
```

```
printf("%d",L);
```



4. Length:

Length

The number of characters in a string is called its length. We will write
`LENGTH(string)`

TRIM omits trailing blanks.

```
#include<stdio.h>
#include<string.h>
main()
{ char S[]=" CSE ", T[10];
  int i=0, j, k;
  j=strlen(S)-1;
  while(S[i]==' ') i++;
  while(S[j]==' ') j--;
  for(k=i; k<=j; k++)
    T[k-i]=S[k];
  T[k-i]='\0';
  printf("%s", T);
}
```

TRIM(' CSE ')	=>	'CSE'
LTRIM(' CSE ')	=>	'CSE '
RTRIM(' CSE ')	=>	' CSE'



Word Processing:

- (a) *Replacement*. Replacing one string in the text by another.
- (b) *Insertion*. Inserting a string in the middle of the text.
- (c) *Deletion*. Deleting a string from the text.

Insertion

Suppose in a given text T we want to insert a string S so that S begins in position K .

`INSERT(text, position, string)`

For example,

`INSERT ('ABCDEFGH', 3, 'XYZ') = 'ABXYZCDEFGH'`

`INSERT ('ABCDEFGH', 6, 'XYZ') = 'ABCDEXYZFGH'`

By String Operation:

$\text{INSERT}(T, K, S) = \text{SUBSTRING}(T, 1, K - 1) // S // \text{SUBSTRING}(T, K, \text{LENGTH}(T) - K + 1)$



Word Processing:

- (a) *Replacement*. Replacing one string in the text by another.
- (b) *Insertion*. Inserting a string in the middle of the text.
- (c) *Deletion*. Deleting a string from the text.

Deletion

Suppose in a given text T we want to delete the substring which begins in position K and has length L . We denote this operation by

$\text{DELETE}(\text{text}, \text{position}, \text{length})$

For example,

$\text{DELETE}(' \text{ABCDEFGFG} ', 4, 2) = ' \text{ABCFG} '$
 $\text{DELETE}(' \text{ABCDEFGFG} ', 2, 4) = ' \text{AFG} '$

We assume that nothing is deleted if position $K = 0$. Thus

$\text{DELETE}(' \text{ABCDEFGFG} ', 0, 2) = ' \text{ABCDEFGFG} '$

Using String Operations:

$\text{DELETE}(T, K, L) =$
 $\text{SUBSTRING}(T, 1, K - 1) // \text{SUBSTRING}(T, K + L, \text{LENGTH}(T) - K - L + 1)$



Word Processing:

- (a) *Replacement*. Replacing one string in the text by another.
- (b) *Insertion*. Inserting a string in the middle of the text.
- (c) *Deletion*. Deleting a string from the text.

Deletion

Suppose in a given text T we want to delete the substring which begins in position K and has length L . We denote this operation by

$\text{DELETE}(\text{text}, \text{position}, \text{length})$

Now suppose text T and pattern P are given and we want to delete from T the first occurrence of the pattern P . This can be accomplished by using the above DELETE function as follows:

$\text{DELETE}(T, \text{INDEX}(T, P), \text{LENGTH}(P))$

Suppose after reading into the computer a text T and a pattern P , we want to delete every occurrence of the pattern P in the text T . This can be accomplished by repeatedly applying

$\text{DELETE}(T, \text{INDEX}(T, P), \text{LENGTH}(P))$



Word Processing:

- (a) *Replacement*. Replacing one string in the text by another.
- (b) *Insertion*. Inserting a string in the middle of the text.
- (c) *Deletion*. Deleting a string from the text.

Deletion

Suppose in a given text T we want to delete the substring which begins in position K and has length L . We denote this operation by

$\text{DELETE}(\text{text}, \text{position}, \text{length})$

- (a) Suppose $T = \text{'ABCDEFGG'}$ and $P = \text{'CD'}$. Then $\text{INDEX}(T, P) = 3$ and $\text{LENGTH}(P) = 2$. Hence

$\text{DELETE}(\text{'ABCDEFGG'}, 3, 2) = \text{'ABEFG'}$

- (b) Suppose $T = \text{'ABCDEFGG'}$ and $P = \text{'DC'}$. Then $\text{INDEX}(T, P) = 0$ and $\text{LENGTH}(P) = 2$. Hence, by the "zero case,"

$\text{DELETE}(\text{'ABCDEFGG'}, 0, 2) = \text{'ABCDEFGG'}$

as expected.



Word Processing:

- (a) *Replacement*. Replacing one string in the text by another.
- (b) *Insertion*. Inserting a string in the middle of the text.
- (c) *Deletion*. Deleting a string from the text.

Algorithm 3.1: A text T and a pattern P are in memory. This algorithm deletes every occurrence of P in T .

1. [Find index of P .] Set $K := \text{INDEX}(T, P)$.
2. Repeat while $K \neq 0$:
 - (a) [Delete P from T .]
Set $T := \text{DELETE}(T, \text{INDEX}(T, P), \text{LENGTH}(P))$
 - (b) [Update index.] Set $K := \text{INDEX}(T, P)$.[End of loop.]
3. Write : T .
4. Exit.



Word Processing:

- (a) *Replacement*. Replacing one string in the text by another.
- (b) *Insertion*. Inserting a string in the middle of the text.
- (c) *Deletion*. Deleting a string from the text.

- (a) Suppose Algorithm 3.1 is run with the data

$$T = \text{XABYABZ}, \quad P = \text{AB}$$

Then the loop in the algorithm will be executed twice. During the first execution, the first occurrence of AB in T is deleted, with the result that $T = \text{XYABZ}$. During the second execution, the remaining occurrence of AB in T is deleted, so that $T = \text{XYZ}$. Accordingly, XYZ is the output.

- (b) Suppose Algorithm 3.1 is run with the data

$$T = \text{XAAABBBY}, \quad P = \text{AB}$$

Observe that the pattern AB occurs only once in T but the loop in the algorithm will be executed three times. Specifically, after AB is deleted the first time from T we have $T = \text{XAABBY}$, and hence AB appears again in T. After AB is deleted a second time from T, we see that $T = \text{XABY}$ and AB still occurs in T. Finally, after AB is deleted a third time from T, we have $T = \text{XY}$ and AB does not appear in T, and thus $\text{INDEX}(T, P) = 0$. Hence XY is the output.



Word Processing:

- (a) *Replacement.* Replacing one string in the text by another.
- (b) *Insertion.* Inserting a string in the middle of the text.
- (c) *Deletion.* Deleting a string from the text.

Replacement

Suppose in a given text T we want to replace the first occurrence of a pattern P_1 by a pattern P_2 . We will denote this operation by

$\text{REPLACE}(\text{text}, \text{pattern}_1, \text{pattern}_2)$

For example,

$\text{REPLACE}(\text{'XABYABZ'}, \text{'AB'}, \text{'C'}) = \text{'XCYABZ'}$
 $\text{REPLACE}(\text{'XABYABZ'}, \text{'BA'}, \text{'C'}) = \text{'XABYABZ'}$

Can ne executed by three steps:

$K := \text{INDEX}(T, P_1)$
 $T := \text{DELETE}(T, K, \text{LENGTH}(P_1))$
 $\text{INSERT}(T, K, P_2)$



Word Processing:

- (a) *Replacement.* Replacing one string in the text by another.
- (b) *Insertion.* Inserting a string in the middle of the text.
- (c) *Deletion.* Deleting a string from the text.

Algorithm 3.2: A text T and patterns P and Q are in memory. This algorithm replaces every occurrence of P in T by Q .

1. [Find index of P .] Set $K := \text{INDEX}(T, P)$.
2. Repeat while $K \neq 0$:
 - (a) [Replace P by Q .] Set $T := \text{REPLACE}(T, P, Q)$.
 - (b) [Update index.] Set $K := \text{INDEX}(T, P)$.[End of loop.]
3. Write: T .
4. Exit.

Run Algorithm 3.2 with

$T = \text{XABYABZ}, \quad P = \text{AB}, \quad Q = \text{C}$



CSE 203: Data Structure

Data
Structure

Word Processing:

- (a) *Replacement.* Replacing one string in the text by another.
- (b) *Insertion.* Inserting a string in the middle of the text.
- (c) *Deletion.* Deleting a string from the text.

Algorithm 3.2: A text T and patterns P and Q are in memory. This algorithm replaces every occurrence of P in T by Q .

1. [Find index of P .] Set $K := \text{INDEX}(T, P)$.
2. Repeat while $K \neq 0$:
 - (a) [Replace P by Q .] Set $T := \text{REPLACE}(T, P, Q)$.
 - (b) [Update index.] Set $K := \text{INDEX}(T, P)$.[End of loop.]
3. Write: T .
4. Exit.

Run Algorithm 3.2 with

$T = XAY, \quad P = A, \quad Q = AB$

Pass 1 $T = XABY$ at the end of the first execution of the loop
Pass 2 $T = XAB^2Y$ at the end of the second execution of the loop



CSE 203: Data Structure

Data
Structure



CSE 203: Data Structure

Data
Structure